```
create domain HourlyWage numeric(5,2)
        constraint wage-value-test check(value >= 4.00)
```

The domain *HourlyWage* has a constraint that ensures that the hourly wage is greater than 4.00. The clause **constraint** *wage-value-test* is optional, and is used to give the name *wage-value-test* to the constraint. The name is used to indicate which constraint an update violated.

The **check** clause can also be used to restrict a domain to not contain any null values:

```
create domain AccountNumber char(10)
        constraint account-number-null-test check(value not null )
```

As another example, the domain can be restricted to contain only a specified set of values by using the **in** clause:

```
create domain AccountType char(10)
        constraint account-type-test
                check(value in ('Checking', 'Saving'))
```

The preceding **check** conditions can be tested quite easily, when a tuple is inserted or modified. However, in general, the **check** conditions can be more complex (and harder to check), since subqueries that refer to other relations are permitted in the **check** condition. For example, this constraint could be specified on the relation *deposit*:

```
check (branch-name in (select branch-name from branch))
```

The **check** condition verifies that the *branch-name* in each tuple in the *deposit* relation is actually the name of a branch in the *branch* relation. Thus, the condition has to be checked not only when a tuple is inserted or modified in *deposit*, but also when the relation *branch* changes (in this case, when a tuple is deleted or modified in relation *branch*).

The preceding constraint is actually an example of a class of constraints called *referential-integrity* constraints. We discuss such constraints, along with a simpler way of specifying them in SQL, in Section 6.2.

Complex **check** conditions can be useful when we want to ensure integrity of data, but we should use them with care, since they may be costly to test.

6.2 Referential Integrity

Often, we wish to ensure that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation. This condition is called **referential integrity**.

6.2.1 Basic Concepts

Consider a pair of relations $r(R)$ and s(S), and the natural join $r \bowtie s$. There may be a tuple t_r in *r* that does not join with any tuple in *s*. That is, there is no t_s in *s* such that $t_r[R \cap S] = t_s[R \cap S]$. Such tuples are called *dangling* tuples. Depending on the entity set or relationship set being modeled, dangling tuples may or may not be acceptable. In Section 3.3.3, we considered a modified form of join—the outer join—to operate on relations containing dangling tuples. Here, our concern is not with queries, but rather with when we should permit dangling tuples to exist in the database.

Suppose there is a tuple t_1 in the *account* relation with $t_1[branch\text{-}name] =$ "Lunartown," but there is no tuple in the *branch* relation for the Lunartown branch. This situation would be undesirable. We expect the *branch* relation to list all bank branches. Therefore, tuple t_1 would refer to an account at a branch that does not exist. Clearly, we would like to have an integrity constraint that prohibits dangling tuples of this sort.

Not all instances of dangling tuples are undesirable, however. Assume that there is a tuple t_2 in the *branch* relation with $t_2[branch\text{-}name] =$ "Mokan," but there is no tuple in the *account* relation for the Mokan branch. In this case, a branch exists that has no accounts. Although this situation is not common, it may arise when a branch is opened or is about to close. Thus, we do not want to prohibit this situation.

The distinction between these two examples arises from two facts:

- The attribute *branch-name* in *Account-schema* is a foreign key referencing the primary key of *Branch-schema*.
- The attribute *branch-name* in *Branch-schema* is not a foreign key.

(Recall from Section 3.1.3 that a foreign key is a set of attributes in a relation schema that forms a primary key for another schema.)

In the Lunartown example, tuple t_1 in *account* has a value on the foreign key *branch-name* that does not appear in *branch*. In the Mokan-branch example, tuple t_2 in *branch* has a value on *branch-name* that does not appear in *account*, but *branch-name* is not a foreign key. Thus, the distinction between our two examples of dangling tuples is the presence of a foreign key.

Let $r_1(R_1)$ and $r_2(R_2)$ be relations with primary keys K_1 and K_2, respectively. We say that a subset α of R_2 is a **foreign key** referencing K_1 in relation r_1 if it is required that, for every t_2 in r_2, there must be a tuple t_1 in r_1 such that $t_1[K_1] = t_2[\alpha]$. Requirements of this form are called **referential integrity constraints**, or **subset dependencies**. The latter term arises because the preceding referential-integrity constraint can be written as $\Pi_\alpha (r_2) \subseteq \Pi_{K_1} (r_1)$. Note that, for a referential-integrity constraint to make sense, either α must be equal to K_1, or α and K_1 must be compatible sets of attributes.

6.2.2 Referential Integrity and the E-R Model

Referential-integrity constraints arise frequently. If we derive our relational-database schema by constructing tables from E-R diagrams, as we did in Chapter 2, then every

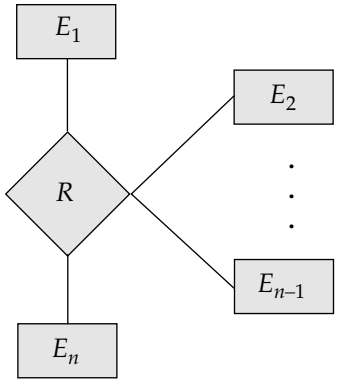


Figure 6.1 An n-ary relationship set.

relation arising from a relationship set has referential-integrity constraints. Figure 6.1 shows an n-ary relationship set R, relating entity sets $E_1, E_2, \ldots, E_n$. Let K_i denote the primary key of E_i. The attributes of the relation schema for relationship set R include $K_1 \cup K_2 \cup \cdots \cup K_n$. The following referential integrity constraints are then present: For each i, K_i in the schema for R is a foreign key referencing K_i in the relation schema generated from entity set E_i

Another source of referential-integrity constraints is weak entity sets. Recall from Chapter 2 that the relation schema for a weak entity set must include the primary key of the entity set on which the weak entity set depends. Thus, the relation schema for each weak entity set includes a foreign key that leads to a referential-integrity constraint.

6.2.3 Database Modification

Database modifications can cause violations of referential integrity. We list here the test that we must make for each type of database modification to preserve the following referential-integrity constraint:

$$\Pi_\alpha (r_2) \subseteq \Pi_K (r_1)$$

- **Insert**. If a tuple t_2 is inserted into r_2, the system must ensure that there is a tuple t_1 in r_1 such that $t_1[K] = t_2[\alpha]$. That is,

  $$t_2[\alpha] \in \Pi_K (r_1)$$

- **Delete**. If a tuple t_1 is deleted from r_1, the system must compute the set of tuples in r_2 that reference t_1:

  $$\sigma_{\alpha = t_1[K]} (r_2)$$

  If this set is not empty, either the delete command is rejected as an error, or the tuples that reference t_1 must themselves be deleted. The latter solution may lead to cascading deletions, since tuples may reference tuples that reference t_1, and so on.

- **Update**. We must consider two cases for update: updates to the referencing relation (r_2), and updates to the referenced relation (r_1).
  - ☐ If a tuple t_2 is updated in relation r_2, and the update modifies values for the foreign key α, then a test similar to the insert case is made. Let t_2' denote the new value of tuple t_2. The system must ensure that
    $$t_2'[\alpha] \in \Pi_K (r_1)$$
  - ☐ If a tuple t_1 is updated in r_1, and the update modifies values for the primary key (K), then a test similar to the delete case is made. The system must compute
    $$\sigma_{\alpha = t_1[K]} (r_2)$$
    using the old value of t_1 (the value before the update is applied). If this set is not empty, the update is rejected as an error, or the update is cascaded in a manner similar to delete.

6.2.4 Referential Integrity in SQL

Foreign keys can be specified as part of the SQL **create table** statement by using the **foreign key** clause. We illustrate foreign-key declarations by using the SQL DDL definition of part of our bank database, shown in Figure 6.2.

By default, a foreign key references the primary key attributes of the referenced table. SQL also supports a version of the **references** clause where a list of attributes of the referenced relation can be specified explicitly. The specified list of attributes must be declared as a candidate key of the referenced relation.

We can use the following short form as part of an attribute definition to declare that the attribute forms a foreign key:

branch-name **char**(15) **references** *branch*

When a referential-integrity constraint is violated, the normal procedure is to reject the action that caused the violation. However, a **foreign key** clause can specify that if a delete or update action on the referenced relation violates the constraint, then, instead of rejecting the action, the system must take steps to change the tuple in the referencing relation to restore the constraint. Consider this definition of an integrity constraint on the relation *account*:

create table *account*
(. . .
foreign key (*branch-name*) **references** *branch*
on delete cascade
on update cascade,
. . .)

Because of the clause **on delete cascade** associated with the foreign-key declaration, if a delete of a tuple in *branch* results in this referential-integrity constraint being violated, the system does not reject the delete. Instead, the delete "cascades" to the

```
create table customer
    (customer-name    char(20),
     customer-street  char(30),
     customer-city    char(30),
     primary key (customer-name))

create table branch
    (branch-name      char(15),
     branch-city      char(30),
     assets           integer,
     primary key (branch-name),
     check (assets >= 0))

create table account
    (account-number   char(10),
     branch-name      char(15),
     balance          integer,
     primary key (account-number),
     foreign key (branch-name) references branch,
     check (balance >= 0))

create table depositor
    (customer-name    char(20),
     account-number   char(10),
     primary key (customer-name, account-number),
     foreign key (customer-name) references customer,
     foreign key (account-number) references account)
```

Figure 6.2 SQL data definition for part of the bank database.

account relation, deleting the tuple that refers to the branch that was deleted. Similarly, the system does not reject an update to a field referenced by the constraint if it violates the constraint; instead, the system updates the field *branch-name* in the referencing tuples in *account* to the new value as well. SQL also allows the **foreign key** clause to specify actions other than **cascade**, if the constraint is violated: The referencing field (here, *branch-name*) can be set to null (by using **set null** in place of **cascade**), or to the default value for the domain (by using **set default**).

If there is a chain of foreign-key dependencies across multiple relations, a deletion or update at one end of the chain can propagate across the entire chain. An interesting case where the **foreign key** constraint on a relation references the same relation appears in Exercise 6.4. If a cascading update or delete causes a constraint violation that cannot be handled by a further cascading operation, the system aborts the transaction. As a result, all the changes caused by the transaction and its cascading actions are undone.

Null values complicate the semantics of referential integrity constraints in SQL. Attributes of foreign keys are allowed to be null, provided that they have not other-